

Data de Execução: 16/03/2025

Semana 6 - Aula 1

Tópico Principal: Linguagens de Programação Back-End

Subtítulo/Tema Específico: Introdução ao Python para Back-End

Código da Aula: [SIS]ANO2C2B1S6A1

Objetivos da Aula:

- Identificar as principais características do Python voltadas ao desenvolvimento back-end.
- Reconhecer a versatilidade da linguagem em áreas como Ciência de Dados e automação.

Recursos Adicionais:

- Computador com acesso à internet.
- Caderno, lápis e caneta para anotações e esboços.
- Recurso audiovisual para exibição de slides.

Exposição Detalhada do Conteúdo

[Slide 5 - Objetivos e Habilidades]

Definição: A aula foca em compreender por que o Python é uma escolha líder para o lado do servidor (back-end), destacando sua sintaxe simples e legibilidade. **Aprofundamento:** Python é uma linguagem interpretada, o que significa que o código é executado linha por linha, facilitando o debug. Sua filosofia (The Zen of Python) preza pela clareza: "Bonito é melhor que feio. Explícito é melhor que implícito". [Informação complementar via busca externa].

[Slide 11 - Estruturas de Dados]

Definição: O Python utiliza estruturas como Dicionários para armazenar dados em pares de **chave-valor**, essencial para manipular formatos como JSON em APIs. **Aprofundamento:** Diferente das listas (ordenadas por índice), os dicionários permitem acesso rápido aos dados através de nomes únicos (chaves). Isso é fundamental no back-end para representar entidades do banco de dados de forma intuitiva. [Informação complementar via busca externa].

[Slide 6 - Colocando em Prática: Consumo de API]

Definição: A atividade prática consiste em criar um script para consumir uma API de previsão do tempo utilizando a biblioteca `requests`. **Exemplo Prático:** Imagine que você está desenvolvendo um aplicativo de entregas. O seu back-end precisa consultar uma API de mapas para calcular a rota e o tempo estimado de chegada. Usando `requests.get()`, seu servidor Python "pergunta" ao mapa a distância e recebe os dados para mostrar ao usuário.

Fechamento da Aula

- **Resumo:** 1. Python é valorizado pela sua legibilidade e vasta gama de bibliotecas prontas. 2. Frameworks como Django e Flask são os pilares para construir servidores robustos com Python. 3. A biblioteca `requests` é a ferramenta padrão para comunicação entre sistemas via HTTP.
- **Curadoria Digital:** [Curso de Python #01 - Seja um Programador](#) - Um excelente ponto de partida para entender a instalação e a lógica inicial da linguagem.
- **Bibliografia:**
 1. RAMALHO, Luciano. **Python Fluente:** Programação concisa, declarativa e eficaz. São Paulo: Novatec, 2015.

2. MATTHES, Eric. **Curso Intensivo de Python**: Uma introdução prática e baseada em projetos à programação. São Paulo: Novatec, 2016.

Semana 6 - Aula 2

Tópico Principal: Frameworks Web

Subtítulo/Tema Específico: Primeiros passos com Django

Código da Aula: [SIS]ANO2C2B1S6A2

Objetivos da Aula:

- Compreender o conceito de framework e o princípio "batteries included" do Django.
- Desenvolver uma aplicação básica de blog utilizando o padrão de projeto do Django.

Recursos Adicionais:

- Ambiente de desenvolvimento Python instalado.
- Navegador web para testar a aplicação.

Exposição Detalhada do Conteúdo

[Slide 2 - Introdução ao Django]

Definição: Django é um framework de alto nível que incentiva o desenvolvimento rápido. Ele segue a filosofia de "baterias inclusas", vindo com sistema de autenticação e painel administrativo prontos. **Aprofundamento:** O Django resolve problemas comuns de segurança automaticamente, como injeção de SQL e Cross-Site Request Forgery (CSRF), permitindo que o desenvolvedor foque na regra de negócio e não em configurações básicas. [Informação complementar via busca externa].

[Slide 7 - Padrão MTV]

Definição: O Django utiliza o padrão **Model-Template-View**. O *Model* cuida dos dados, a *View* da lógica e o *Template* da apresentação. **Aprofundamento:** Embora seja similar ao famoso MVC (Model-View-Controller), no Django a "View" atua como o controlador que processa a requisição, e o "Template" é o que seria a View no MVC tradicional. [Informação complementar via busca externa].

[Slide 6 - Prática: Aplicação de Blog]

Definição: Criação de um modelo de postagem com título, conteúdo e data, seguido da configuração de rotas (URLs) e exibição em HTML. **Exemplo Prático:** Ao acessar um portal de notícias como o G1, o Django (ou framework similar) busca no banco de dados o "Model" da notícia, processa os dados na "View" e renderiza o texto e fotos no "Template" que você vê na tela.

Fechamento da Aula

- **Resumo:**
 1. Django foca em produtividade e organização através do padrão MTV.
 2. O framework automatiza tarefas complexas como a gestão de bancos de dados via Models.
 3. A estrutura de URLs do Django permite criar endereços amigáveis e organizados para o usuário.

- **Curadoria Digital:** [Estrutura Básica de um Projeto em Django](#) - Tutorial passo a passo para configurar seu primeiro projeto Django.
- **Bibliografia:**
 1. MELLO, Fernando. **Django**: Desenvolvimento Web Rápido e Limpo com Python. São Paulo: Casa do Código, 2018.
 2. FORCIER, Jeff; BISSEX, Paul; CHUN, Wesley. **Python Web Development with Django**. Upper Saddle River: Addison-Wesley, 2008.

Semana 6 - Aula 3

Tópico Principal: Alternativas de Back-End

Subtítulo/Tema Específico: Primeiros passos com C#

Código da Aula: [SIS]ANO2C2B1S6A3

Objetivos da Aula:

- Identificar as características da linguagem C# e sua aplicação no ecossistema .NET.
- Aplicar conceitos de Programação Orientada a Objetos (POO) em uma aplicação console.

Recursos Adicionais:

- Visual Studio ou Visual Studio Code.
- SDK do .NET instalado.

Exposição Detalhada do Conteúdo

[Slide 2 - Introdução ao C#]

Definição: Desenvolvida pela Microsoft, C# é uma linguagem moderna, fortemente tipada e orientada a objetos, amplamente usada para back-end empresarial e jogos. **Aprofundamento:** C# roda sobre o CLR (Common Language Runtime), que gerencia a memória automaticamente (Garbage Collection), evitando erros comuns de vazamento de memória encontrados em linguagens como C++. [Informação complementar via busca externa].

[Slide 7 - Orientação a Objetos]

Definição: A aula introduz classes, encapsulamento e herança, usando o exemplo de um gerenciador de tarefas. **Aprofundamento:** Em C#, tudo é um objeto. Uma classe funciona como uma "planta arquitetônica", e o objeto é a casa construída. O encapsulamento protege os dados internos do objeto de acessos indevidos através de modificadores como `private` e `public`. [Informação complementar via busca externa].

[Slide 6 - Prática: App de Tarefas]

Definição: Desenvolvimento de uma aplicação que permite adicionar e listar tarefas com status de "Pendente" ou "Concluída". **Exemplo Prático:** No sistema de um banco, cada "Conta Corrente" é um objeto de uma classe. Quando você saca dinheiro, o sistema chama um método `Sacar()` que altera o saldo (propriedade) daquele objeto específico, garantindo que o dinheiro saia da conta certa.

Fechamento da Aula

- **Resumo:**

1. C# é a linguagem base para o ecossistema .NET, ideal para sistemas de larga escala.
2. O domínio da Programação Orientada a Objetos é fundamental para organizar códigos complexos.
3. O Visual Studio é a principal IDE que oferece ferramentas de depuração poderosas para C#.

- **Curadoria Digital:** [C# para Iniciantes - O que é e por onde começar](#) - Vídeo didático sobre as possibilidades da carreira com C#.

- **Bibliografia:**

1. DEITEL, Paul; DEITEL, Harvey. **C# 6 para Programadores**. Rio de Janeiro: Alta Books, 2017.
2. ALBAHARI, Joseph. **C# 10 in a Nutshell: The Definitive Reference**. Sebastopol: O'Reilly Media, 2022.